239. Sliding Window Maximum

```c
#include <stdlib.h>

int* maxSlidingWindow(int* nums, int numsSize, int k, int* returnSize) {
    if (numsSize == 0 || k == 0) {
        *returnSize = 0;
        return NULL;
    }

    *returnSize = numsSize - k + 1;
    int* result = (int*)malloc((*returnSize) * sizeof(int));
    int* deque = (int*)malloc(numsSize * sizeof(int));

    int head = 0;
    int tail = 0;
    int resIdx = 0;

    for (int i = 0; i < numsSize; i++) {
        if (head < tail && deque[head] < i - k + 1) {
            head++;
        }

        while (head < tail && nums[deque[tail - 1]] <= nums[i]) {
            tail--;
        }

        deque[tail++] = i;

        if (i >= k - 1) {
            result[resIdx++] = nums[deque[head]];
        }
    }

    free(deque);
    return result;
}
```

480. Sliding Window Median

```c
#include <stdlib.h>
#include <string.h>

int compareLong(const void* a, const void* b) {
    long long va = *(const long long*)a;
    long long vb = *(const long long*)b;
    return (va > vb) - (va < vb);
}
```

```c
int binarySearch(long long* arr, int size, long long val) {
    int low = 0, high = size - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == val) return mid;
        if (arr[mid] < val) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
```

```c
double* medianSlidingWindow(int* nums, int numsSize, int k, int* returnSize) {
```

```
    *returnSize = numsSize - k + 1;
    double* result = (double*)malloc((*returnSize) * sizeof(double));
    long long* window = (long long*)malloc(k * sizeof(long long));

    for (int i = 0; i < k; i++) {
        window[i] = nums[i];
    }

    qsort(window, k, sizeof(long long), compareLong);

    int isEven = (k % 2 == 0);
    int mid1 = (k - 1) / 2;
    int mid2 = k / 2;

    result[0] = isEven ? ((double)window[mid1] + window[mid2]) / 2.0 : (double)window[mid1];

    for (int i = k; i < numsSize; i++) {
        int oldVal = nums[i - k];
        int newVal = nums[i];

        int delIdx = binarySearch(window, k, oldVal);

        int insIdx = 0;
        int low = 0, high = k - 1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (window[mid] < newVal) {
                insIdx = mid + 1;
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }

        if (delIdx < insIdx) {
            memmove(&window[delIdx], &window[delIdx + 1], (insIdx - delIdx - 1) * sizeof(long long));
            window[insIdx - 1] = newVal;
        } else {
            memmove(&window[insIdx + 1], &window[insIdx], (delIdx - insIdx) * sizeof(long long));
            window[insIdx] = newVal;
        }

        result[i - k + 1] = isEven ? ((double)window[mid1] + window[mid2]) / 2.0 :
(double)window[mid1];
    }

    free(window);
    return result;
}
```